

CONNECTIONS BETWEEN SCHEDULE-FREE OPTIMIZERS, ADE-MAMIX, AND ACCELERATED SGD VARIANTS

Anonymous authors
Anonymous submission

ABSTRACT

Two lines of work in deep learning optimization have so far developed in parallel: a wave of new optimizers—Schedule-Free methods, AdEMAMix, MARS, and Lion—that all rework the classical momentum mechanism, and a body of theory that accelerates stochastic gradient descent (SGD) in the noise-dominated regime by decoupling the momentum coefficient from the weight placed on the current gradient. This paper makes three contributions that tie these threads together. First, we prove explicit equivalences showing that these recent optimizers are, at their core, instances of accelerated SGD (optionally combined with preconditioning or weight averaging). Second, we validate the resulting predictions empirically on a 150m-parameter language modeling task, where AdEMAMix—the method that most closely matches accelerated SGD—delivers the strongest performance, exactly as the theory anticipates. Third, building on this understanding, we introduce Simplified-AdEMAMix, a variant that matches AdEMAMix across both large and small batch-size settings while removing the need for two separate momentum terms. Together, these results reframe several recent optimizers as a single, well-understood family and turn that insight into a simpler, equally effective algorithm. The code for Simplified-AdEMAMix will be released upon acceptance.

1 INTRODUCTION

Deep learning optimization has recently seen a surge of new algorithms, including Lion (Chen et al., 2023), Schedule-FreeSGD/AdamW (Defazio et al., 2024), and AdEMAMix (Pagliardini et al., 2024). These optimizers were proposed from distinct motivations, yet they share one defining trait: each reworks the momentum scheme used in the update. This shared trait is the starting point for our paper, and it is what lets us treat them as members of a single family rather than as isolated heuristics.

A separate body of theory has, in parallel, sought to accelerate gradient descent under noise. Classical momentum—heavy-ball or Nesterov—accelerates deterministic gradient descent (especially for quadratics) but provably fails to accelerate SGD (Jain et al., 2018; Liu & Belkin, 2020). That gap prompted a line of alternative momentum schemes designed to achieve acceleration in the presence of noise (Jain et al., 2018; Vaswani et al., 2019; Liu & Belkin, 2020; Gupta et al., 2023). The key structural insight from this literature, which we build on throughout, is that every proposed accelerated SGD method can be read as decoupling the momentum coefficient from the weight assigned to the current gradient in the update.

The central contribution of this paper is to connect these two research directions directly. Concretely, we prove that Schedule-Free SGD is mathematically equivalent to accelerated SGD followed by weight averaging, and we show that optimizers such as Lion, Schedule-Free AdamW, and AdEMAMix can be understood as accelerated SGD combined with preconditioning. While fragments of these connections have appeared in prior work (Defazio, 2021), to the best of our knowledge the relationship between this new generation of optimizers and accelerated SGD has not been formally established before. Establishing it is what makes the rest of our contributions possible.

We then put these theoretical predictions to the test on a 150m decoder-only transformer, trained on 15b tokens with a small batch size of 32k tokens so that training stays in the noise-dominated regime. The experiments confirm the theory on every point we can check: Schedule-Free AdamW tracks accelerated SGD-based AdamW (Algorithm 3) closely, accelerated methods give a modest edge at small batch sizes, and—consistent with the same theory—these benefits vanish once the batch size grows large. This tight agreement between prediction and measurement is itself part of what we contribute: it turns the equivalences from a formal curiosity into an actionable design principle.

We summarize our main contributions below:

1. **A unifying theory.** We establish precise theoretical connections between accelerated SGD and recently proposed optimizers such as Schedule-Free SGD and AdEMAMix, placing several seemingly distinct methods under one framework.
2. **Empirical validation of the theory.** On a 150-million-parameter decoder-only transformer, we compare AdamW, Schedule-Free AdamW, AdEMAMix, and MARS, and find that AdEMAMix—the method that most closely aligns with accelerated SGD variants—performs best, exactly as the framework predicts.
3. **A batch-size boundary on the benefit.** As anticipated from the equivalence to accelerated SGD, the advantages of these methods diminish at large batch sizes relative to Adam; concretely, we show that Adam with momentum scheduling already matches AdEMAMix in this regime.
4. **An explanation for Schedule-Free’s large-batch failure.** At high batch sizes, Schedule-Free AdamW performs markedly worse than AdamW with cosine decay, and we trace this to the intrinsic coupling of the momentum and weight-averaging coefficients in Schedule-Free optimizers.
5. **A simpler algorithm.** We introduce Simplified-AdEMAMix, a modification that preserves AdEMAMix’s performance across both large and small batch-size regimes while eliminating the need for two distinct momentum terms.

2 RELATED WORK

Our contributions sit at the intersection of two literatures—the theory of accelerated SGD and the recent wave of practical optimizers—so we review the parts of each that our connections rely on.

On the theory side, Jain et al. (2018) introduced an accelerated SGD variant with improved convergence rates for least-squares. Kidambi et al. (2018) then simplified its update rule and formally proved that momentum does not provide acceleration in this case, sharpening the very distinction our framework exploits. Subsequent works (Liu & Belkin, 2020; Vaswani et al., 2019; Gupta et al., 2023) extended these guarantees to general convex and strongly convex functions under various assumptions.

On the practical side, several optimizers have, over the years, drifted toward the structure of these accelerated SGD variants—a convergence that our paper makes explicit. Lucas et al. (2019) combined a weighted sum of multiple momentum terms with distinct coefficients to form the update. Ma & Yarats (2019) designed an optimizer explicitly inspired by the framework of Jain et al. (2018). More recently, Chen et al. (2023) obtained an optimizer via genetic search that, like earlier accelerated SGD variants, assigns different weights to the gradient and the momentum term, and Pagliardini et al. (2024) proposed blending two distinct momentum scales in the final update. Recognizing that these methods are all variations on accelerated SGD is precisely the gap this paper closes.

3 BACKGROUND

3.1 MOMENTUM

We first fix the three ingredients—momentum, weight averaging, and accelerated SGD—that our connections are built from. Momentum is a well-established technique for accelerating the convergence of gradient descent in deterministic settings. The momentum update for weights w_t , with a momentum coefficient β , is given by:

$$m_t = \beta m_{t-1} + \nabla f(w_t); \quad w_t = w_{t-1} - \eta m_t$$

3.2 WEIGHT AVERAGING

Weight averaging is a widely used technique in stochastic optimization for reducing noise in the iterates, and it is the second ingredient our Schedule-Free analysis relies on. Instead of returning the final iterate w_T , one computes a weighted average $\bar{w}_T$ of the iterates, with weights denoted by γ_t :

$$\bar{w}_T = (1 - \gamma_T)\bar{w}_{T-1} + \gamma_T w_T$$

All instances of weight averaging in this paper utilize coefficients γ_t of the form:

$$\gamma_t \approx 1 - \frac{1}{\delta t}$$

for some constant $0 \leq \delta \leq 1$.

3.3 ACCELERATED SGD

The third ingredient—and the unifying object of the paper—is accelerated SGD. We state it as a single generalized framework that encompasses many accelerated SGD methods, and to which we will reduce each optimizer in the sections that follow:

$$m_t = \beta_{a,t}m_{t-1} + g_t, \quad w_{t+1} = w_t - \eta_{a,t}m_t - \alpha_{a,t}g_t \quad (1)$$

where $\beta_{a,t}, \alpha_{a,t}, \eta_{a,t}$ are (possibly time-dependent) scalar coefficients, and g_t represents the stochastic gradient evaluated at w_t . We use the subscript ‘a’ to indicate coefficients that adhere to this specific accelerated SGD formulation.

Two observations make this framework the right lens for the rest of the paper. First, setting $\alpha_{a,t} = 0$ recovers standard SGD with momentum, so acceleration is captured entirely by the extra $\alpha_{a,t}g_t$ term—the decoupling highlighted in the introduction. Second, and as observed in prior work, many accelerated SGD algorithms in the literature—such as those of Jain et al. (2018); Vaswani et al. (2019); Liu & Belkin (2020); Gupta et al. (2023)—fall directly within this single form; we give the precise reductions in Appendix B. Having a common form is what allows the equivalences in the next section to be stated cleanly.

4 CONNECTIONS BETWEEN EXISTING OPTIMIZERS AND ACCELERATED SGD

This section delivers the paper’s first contribution in detail: we show, one optimizer at a time, that Schedule-Free methods, Lion, MARS, and AdEMAMix are each an instance of the accelerated SGD framework of Equation (1), optionally wrapped in weight averaging or preconditioning. These reductions are not merely descriptive—they directly motivate our fifth contribution, a simplified variant of AdEMAMix that uses a single momentum term yet matches AdEMAMix across both small and large batch-size regimes.

4.1 SCHEDULE-FREE SGD

We begin with Schedule-Free SGD, our cleanest equivalence. Schedule-Free SGD (Defazio et al., 2024) is a recently introduced constant-learning-rate optimizer designed to eliminate the need for scheduling. Following the notation of Defazio et al. (2024), its update equations are:

$$\begin{aligned} y_t &= (1 - \beta)z_t + \beta x_t \\ z_{t+1} &= z_t - \gamma g(y_t) \\ x_{t+1} &= (1 - c_{t+1})x_t + c_{t+1}z_{t+1} \end{aligned}$$

Here, y_t represents the current model weights (where the gradient is evaluated), while x_t denotes the weights used for evaluation. Our goal is to show that y_t obeys the accelerated SGD update and that x_t is merely a weighted average of the y_t trajectory. We first express the update in terms of y_t and m_t , defining:

$$m_{t+1} = \frac{x_t - z_{t+1}}{\gamma}. \quad (2)$$

Further simplifying m_{t+1} , we obtain:

$$m_t = \frac{x_t - z_{t+1}}{\gamma} \quad (3)$$

$$= \frac{x_t + \gamma g_t - z_t}{\gamma} \quad (4)$$

$$= \frac{x_t - z_t + \gamma g_t}{\gamma} \quad (5)$$

$$= \frac{(1 - c_t)(x_{t-1} - z_t) + \gamma g_t}{\gamma} \quad (6)$$

$$= (1 - c_t)m_{t-1} + g_t. \quad (7)$$

Thus, m_t follows the momentum update in Equation (1) with $\beta_{a,t} = 1 - c_t$. Given m_t , we now examine the update for y_t :

$$y_{t+1} = (1 - \beta)z_{t+1} + \beta x_{t+1} \quad (8)$$

$$= (1 - \beta)(z_t - \gamma g_t) + \beta((1 - c_{t+1})x_t + c_{t+1}z_{t+1}) \quad (9)$$

$$= (1 - \beta)z_t + \beta x_t - (1 - \beta)\gamma g_t + \beta c_{t+1}(z_{t+1} - x_t) \quad (10)$$

$$= y_t - \gamma[\beta c_{t+1}m_t + (1 - \beta)g_t]. \quad (11)$$

Thus, y_t follows the weight update in Equation (1) with $\eta_{a,t} = \gamma\beta c_{t+1}$ and $\alpha_{a,t} = \gamma(1 - \beta)$, where $w_t = y_t$. Consequently, y_t in Schedule-Free SGD precisely follows the accelerated SGD framework. However, x_t is used for evaluation in Schedule-Free SGD. We now analyze the dynamics of x_t :

$$x_{t+1} = (1 - c_{t+1})x_t + c_{t+1}z_{t+1}$$

$$x_{t+1} = (1 - c_{t+1})x_t + c_2 \left(\frac{y_{t+1} - \beta x_{t+1}}{1 - \beta} \right)$$

$$x_{t+1}(1 - \beta + c_{t+1}\beta) = (1 - c_{t+1})(1 - \beta)x_t + c_{t+1}y_{t+1}$$

$$x_{t+1} = \frac{(1 - c_{t+1})(1 - \beta)x_t + c_{t+1}y_{t+1}}{(1 - c_{t+1})(1 - \beta) + c_{t+1}}.$$

Thus x_t is a weighted average of y_t , and recursively expanding it confirms an exponential average of y_t when c_t is constant. This completes the equivalence: *Schedule-Free SGD is exactly accelerated SGD followed by weight averaging*—the first concrete instance of our unifying claim.

This decomposition also explains where Schedule-Free SGD’s benefits come from, separating them into two distinct components:

1. Improved performance compared to standard SGD with momentum, due to its equivalence to accelerated SGD.
2. The ability to use a constant learning rate without scheduling, enabled by weight averaging (specifically, tailed weight averaging; see Section 4.1.3).

Read through this lens, Schedule-Free SGD/Adam enjoys two advantages that are unique to it:

- it requires no additional memory for weight averaging;
- it removes the explicit weight-averaging coefficient as a hyperparameter.

The same lens, however, exposes a cost: because the two coefficients are coupled by construction, this convenience does not come for free. We show in Section 5.1 that precisely this coupling of momentum and weight averaging fails to scale to large batch sizes—the mechanism behind our fourth contribution.

4.1.1 CASE: $\beta = 0.0$

As noted in (Defazio et al., 2024), when $\beta = 0$, Schedule-Free SGD reduces to standard SGD with weight averaging. Since $c_t = 1/t$, it applies weight averaging from the beginning.

4.1.2 CASE: $\beta = 1.0$

As noted in (Defazio et al., 2024), when $\beta = 1$, Schedule-Free SGD reduces to standard momentum SGD, with the momentum coefficient $\beta_{a,t}$ scaling as $1 - 1/t$.

4.1.3 CASE: $\beta = 0.9$

For $\beta = 0.9$, the default setting in Schedule-Free SGD:

- As c_t scales as $1/t$, momentum grows as $1 - 1/t$.
- The ratio of the weight assigned to the current gradient versus momentum is fixed at $(1 - \beta)/(\beta c_{t+1}) \approx 0.11$.
- Weight averaging is applied approximately over the most recent 10% of the iterates.

4.2 LION

Lion fits the same framework with essentially no work. Its update rule (Chen et al., 2023) is:

$$m'_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$

$$\theta_t = \theta_{t-1} - \eta \text{sign}(m'_t)$$

$$m_t = \beta_2 m_{t-1} + (1 - \beta_2) g_t.$$

Reading off the update, Lion (Chen et al., 2023) is simply an accelerated SGD method followed by a coordinate-wise sign operation—another member of the family.

4.3 MARS

MARS requires a short derivation but lands in the same place. Here we show that the practical version of the recently proposed optimizer MARS (Yuan et al., 2024), referred to as MARS-Approx, follows the accelerated SGD framework supplemented by a preconditioning step. Its update equations (ignoring bias correction and clipping) are:

$$\begin{aligned} c_t &= g_t + \gamma \frac{\beta_1}{1 - \beta_1} [g_t - g_{t-1}] \\ m_t &= \beta_1 m_{t-1} + (1 - \beta_1) c_t \\ v_t &= \beta_2 v_{t-1} + (1 - \beta_2) c_t^2 \\ x_{t+1} &= x_t - \eta \frac{m_t}{\sqrt{v_t} + \epsilon} \end{aligned}$$

where m_t and v_t represent the first- and second-order momentum terms, respectively, and x_t denotes the model parameters. Rewriting the update using $\hat{m}_t = m_t - \gamma g_t$, we obtain:

$$\begin{aligned}
c_t &= g_t + \gamma \frac{\beta_1}{1 - \beta_1} [g_t - g_{t-1}] \\
\hat{m}_t &= \beta_1 \hat{m}_{t-1} + (1 - \beta_1)(1 - \gamma)g_t \\
v_t &= \beta_2 v_{t-1} + (1 - \beta_2)c_t^2 \\
x_{t+1} &= x_t - \eta \frac{\hat{m}_t + \gamma g_t}{\sqrt{v_t} + \epsilon}
\end{aligned}$$

This rewriting makes the point plain: the momentum update again follows the general accelerated SGD framework, with MARS differing from AdamW only in its distinct preconditioning. We revisit what this buys MARS empirically in Section 5.

4.4 ADEMAMIX

AdEMAMix is the most consequential case, because it is the method that aligns most tightly with accelerated SGD and, as our experiments will show, performs best. AdEMAMix (Pagliardini et al., 2024) is structurally close to accelerated SGD-based AdamW, differing in that it maintains two distinct momentum terms with different coefficients and combines them linearly, rather than combining the current gradient with a single momentum term. The algorithm is stated in Algorithm 1.

To make the correspondence exact, we analyze the variant of AdEMAMix with $\beta_1 = 0$. Pagliardini et al. (2024) report that this variant is nearly equivalent to the full version at small batch sizes, and our experiments in Section 5 confirm it. With $\beta_1 = 0$, AdEMAMix falls squarely within the accelerated SGD framework of Equation (1). Going further, we show that AdEMAMix’s prescribed schedules for β_3 (the momentum coefficient) and α (the relative weight on the current gradient) closely match the theoretical schedules derived for accelerated SGD (Gupta et al., 2023)—a quantitative match, not just a structural one.

Concretely, in smooth convex optimization, acceleration in stochastic settings requires a momentum scheme of the form:

$$\beta_{a,t} = 1 - \frac{k}{t}$$

for some constant $k > 0$, as established by Gupta et al. (2023), and AdEMAMix approximately follows exactly this scheme by scaling up β_3 . The weighting of the current gradient matches too, as we now check. In accelerated SGD schemes, momentum is maintained in the standard form:

$$m_t = \beta_{a,t} m_{t-1} + g_t$$

whereas in Algorithm 1, the momentum update follows:

$$m_2^{(t)} \leftarrow \beta_3^{(t)} m_2^{(t-1)} + (1 - \beta_3^{(t)}) g^{(t)}.$$

When $\beta_{a,t}$ scales as $1 - 1/t$, the accumulated contribution of past gradients in m_t grows proportionally to t , and AdEMAMix’s coefficient α scales the same way. This close alignment on both the momentum schedule and the gradient weighting is, we argue, why AdEMAMix outperforms the other optimizers in Figure 1—directly supporting our second contribution.

The same analysis also predicts where AdEMAMix should struggle. At large batch sizes, AdEMAMix degrades when $\beta_1 = 0.0$, as reported by Pagliardini et al. (2024): the theory of Gupta et al. (2023) says the weight on the current gradient must shrink as batch size grows, yet AdEMAMix holds that weight fixed at 1. This mismatch is the likely cause of its large-batch decline, and it points directly at a fix.

That fix is the subject of the next section, where we introduce a simplified variant of AdEMAMix that (i) places an explicit, tunable weight on the current gradient, (ii) removes the need to schedule α , and (iii) keeps only a single

momentum term. We then validate empirically that this simplified version matches AdEMAMix across both small and large batch setups.

Algorithm 1 Single step of AdEMAMix optimizer.

- 1: **Input:** Data distribution $\mathcal{D}$. Initial model parameters $\theta^{(0)}$. Number of iterations T . Learning rate η . ϵ a small constant. AdamW parameters: β_1, β_2 . AdEMAMix parameters β_3, α . Warmup parameter T_{α, β_3} , note that we usually set it to T . β_{start} is usually set to β_1 .
 - 2: Optional: use schedulers $\eta^{(t)}, \beta_3^{(t)} \leftarrow f_{\beta_3}(t, \beta_3, \beta_{\text{start}}, T_{\alpha, \beta_3})$ and $\alpha^{(t)} \leftarrow f_{\alpha}(t, \alpha, T_{\alpha, \beta_3})$
 - 3: Sample batch: $x \sim \mathcal{D}$
 - 4: Compute gradient: $g^{(t)} \leftarrow \nabla_{\theta} \mathcal{L}_{\theta^{(t-1)}}(x)$
 - 5: Update the fast EMA m_1 : $m_1^{(t)} \leftarrow \beta_1 m_1^{(t-1)} + (1 - \beta_1) g^{(t)}$
 - 6: Update the slow EMA m_2 : $m_2^{(t)} \leftarrow \beta_3^{(t)} m_2^{(t-1)} + (1 - \beta_3^{(t)}) g^{(t)}$
 - 7: Update the second moment estimate: $\nu^{(t)} \leftarrow \beta_2 \nu^{(t-1)} + (1 - \beta_2) (g^{(t)})^2$
 - 8: Update parameters: $\theta^{(t)} \leftarrow \theta^{(t-1)} - \eta^{(t)} \left(\frac{\hat{m}_1^{(t)} + \alpha^{(t)} m_2^{(t)}}{\sqrt{\hat{\nu}^{(t)}} + \epsilon} \right)$
-

Algorithm 2 Single step of Simplified AdEMAMix optimizer.

- 1: **Input:** Data distribution $\mathcal{D}$. Initial model parameters $\theta^{(0)}$. Number of iterations T . Learning rate η . ϵ a small constant. AdamW parameters: β_1, β_2 . AdEMAMix parameters $\alpha, \beta_{\text{start}}$. Warmup parameter T_{β_1} , note that we usually set it to T .
 - 2: Optional: use schedulers $\eta^{(t)}, \beta_1^{(t)} \leftarrow f_{\beta_1}(t, \beta_1, \beta_{\text{start}}, T_{\beta_1})$
 - 3: Sample batch: $x \sim \mathcal{D}$
 - 4: Compute gradient: $g^{(t)} \leftarrow \nabla_{\theta} \mathcal{L}_{\theta^{(t-1)}}(x)$
 - 5: Update the EMA m_1 : $m_1^{(t)} \leftarrow \beta_1^{(t)} m_1^{(t-1)} + g^{(t)}$
 - 6: Update the second moment estimate: $\nu^{(t)} \leftarrow \beta_2 \nu^{(t-1)} + (1 - \beta_2) (g^{(t)})^2$
 - 7: Update parameters: $\theta^{(t)} \leftarrow \theta^{(t-1)} - \eta^{(t)} \left(\frac{m_1^{(t)} + \alpha g^{(t)}}{\sqrt{\hat{\nu}^{(t)}} + \epsilon} \right)$
-

4.5 SIMPLIFIED ADEMAMIX

We now present the paper’s fifth contribution: Simplified-AdEMAMix, an optimizer that turns the analysis above into a leaner algorithm by removing both the second momentum term and the scheduling of α . It is stated in Algorithm 2, where we use theory-style momentum in place of the exponential-moving-average (EMA) style and assign a single fixed weight α to the current gradient in the final update. Two properties make the design point clear: setting $\alpha = 0$ recovers standard Adam (up to appropriate transformations of η and β_1), so Simplified-AdEMAMix is a strict generalization; and, as we show in Section 5, it matches AdEMAMix across both small and large batch sizes despite carrying only one momentum buffer.

5 EXPERIMENTS

This section tests the theory’s predictions and thereby establishes our second contribution. Our experiments use a 150-million-parameter decoder-only transformer trained for language modeling on the C4 dataset, with a sequence length of 1024 and a batch size of 32, over 15 billion tokens ($\approx 5 \times$ Chinchilla); the small batch size keeps training firmly in the noise-dominated regime where accelerated SGD is expected to help.

To probe the predictions from every relevant angle, we compare the following algorithms:

1. Standard AdamW with cosine decay
2. Standard AdamW with weight averaging

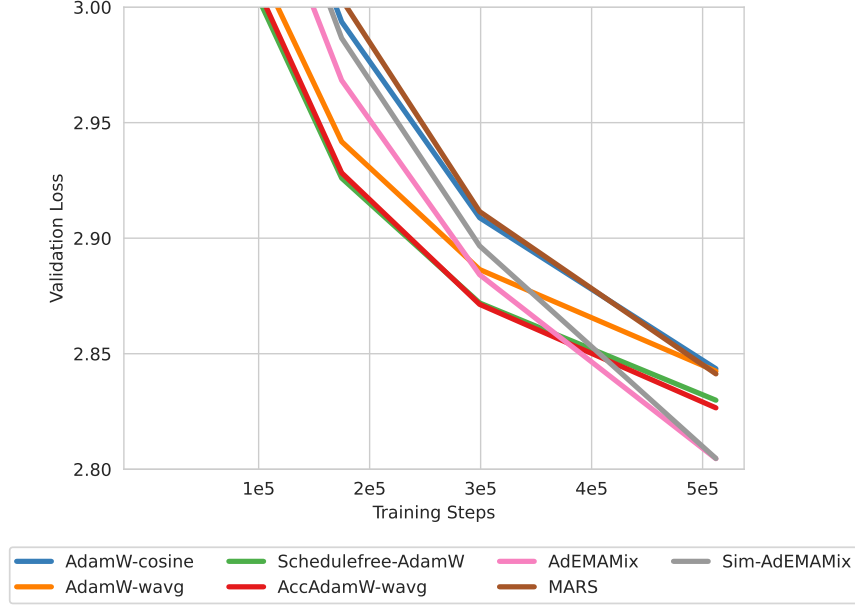


Figure 1: Best runs of the optimizers listed in Section 5 on the 150m decoder-only transformer language-modeling task. The takeaway supporting our second contribution: AdEMAMix and Simplified-AdEMAMix perform best, which we attribute to their close correspondence with accelerated SGD variants.

Algorithm 3 Single step of accelerated SGD based Adam with weight averaging. For simplicity we ignore the initialization, other boundary effects such as bias correction, and weight decay. Hyperparameters: Learning rate η , betas = $(\beta_1, \beta_2, \beta_3)$, weight averaging coefficient δ , and epsilon ϵ .

- 1: Sample batch B_t .
- 2: $g \leftarrow -\nabla_w \phi_{B_t}(w_t)$
- 3: $v \leftarrow \beta_2 v + (1 - \beta_2)(g \odot g)$
- 4: $N \leftarrow \frac{\beta_3 m + (1 - \beta_3)g}{\sqrt{\hat{v}} + \epsilon}$
- 5: $w \leftarrow w - \eta N$
- 6: $m \leftarrow \beta_1 m + (1 - \beta_1)g$
- 7: $c = \max(1 - 1/t, 1 - 1/(\delta t))$
- 8: $w_{\text{avg}} \leftarrow c w_{\text{avg}} + (1 - c)w$

3. Schedule-Free AdamW
4. Accelerated AdamW with weight averaging (Algorithm 3)
5. MARS
6. AdEMAMix
7. Simplified-AdEMAMix

Details of the hyperparameter sweeps for these algorithms are provided in Section A.

The results in Figure 1 confirm each prediction in turn. Schedule-Free AdamW and Accelerated AdamW with tailed weight averaging perform comparably—direct empirical support for our proven equivalence between the two. Both, in turn, outperform AdamW with cosine decay and AdamW with tailed weight averaging. Most importantly, AdEMAMix and Simplified-AdEMAMix outperform all other methods, which we attribute to their being the closest match to accelerated SGD—the central empirical finding of the paper.

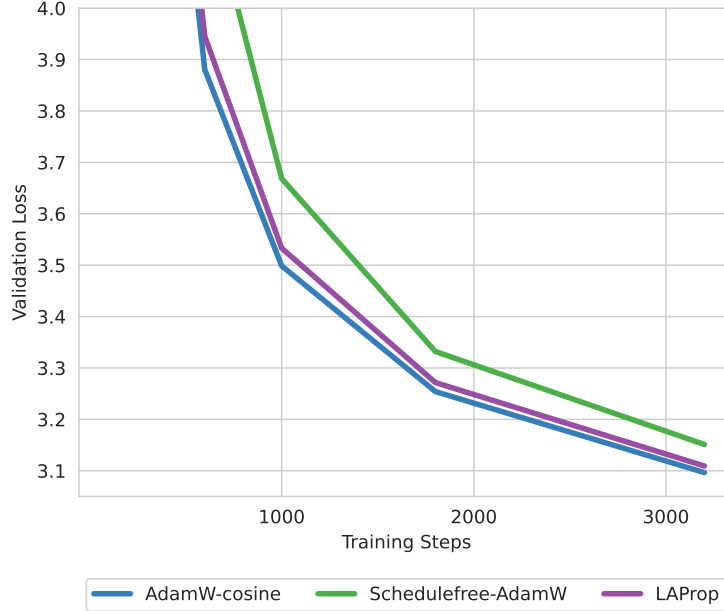


Figure 2: Best runs of AdamW with cosine decay, Schedule-Free AdamW, and LARProp at the higher batch size. This is the evidence for our fourth contribution: Schedule-Free AdamW falls behind AdamW, while LARProp stays comparable, isolating coefficient coupling—not preconditioning order—as the cause. Experimental details are in Section 5.1.

5.1 LARGE BATCH SIZE EXPERIMENTS

Having confirmed the small-batch predictions, we now turn to the large-batch regime, which delivers our third and fourth contributions and stress-tests Simplified-AdEMAMix. Here the theory makes a sharp prediction—the acceleration benefit should disappear—so this regime is exactly where the framework is most at risk. We train with a batch size of 1 million tokens over 3 billion tokens ($\approx$ Chinchilla scale).

Schedule-Free AdamW (our fourth contribution): Figure 2 shows Schedule-Free AdamW falling significantly behind AdamW—and our equivalence explains why. Because Schedule-Free couples the weight-averaging and momentum coefficients, it cannot independently lower momentum, yet at higher batch sizes the optimal momentum is far below $1 - 1/t$. One could adopt a scaling $\approx 1 - r/t$ with $r \geq 1$, but a larger r shrinks the effective weight-averaging window, so the coupling forces a trade-off that AdamW simply does not face.

We also rule out the obvious alternative explanation. AdamW and Schedule-Free AdamW differ in the order of momentum and preconditioning—AdamW applies momentum first, whereas Schedule-Free applies preconditioning first, making the latter algorithmically close to LARProp (Ziyin et al., 2021). If ordering were the culprit, LARProp should lag too; but Figure 2 shows LARProp performing comparably to AdamW, which isolates coefficient coupling—not preconditioning order—as the cause of the gap.

AdEMAMix (our third and fifth contributions): At large batch sizes, and consistent with Pagliardini et al. (2024), Figure 3 shows that setting $\beta_1 = 0.0$ in AdEMAMix causes a significant drop relative to using two separate momentum terms. Our framework names the reason: AdEMAMix pins a fixed weight of 1 on the current gradient, whereas accelerated SGD theory (Gupta et al., 2023) requires that weight to diminish as batch size grows—the same failure mechanism we predicted.

The figure also delivers the payoff. Our Simplified-AdEMAMix matches full AdEMAMix while carrying only a single momentum term, confirming the fifth contribution. Strikingly, it does so at $\alpha = 0.0$, meaning it reduces to standard

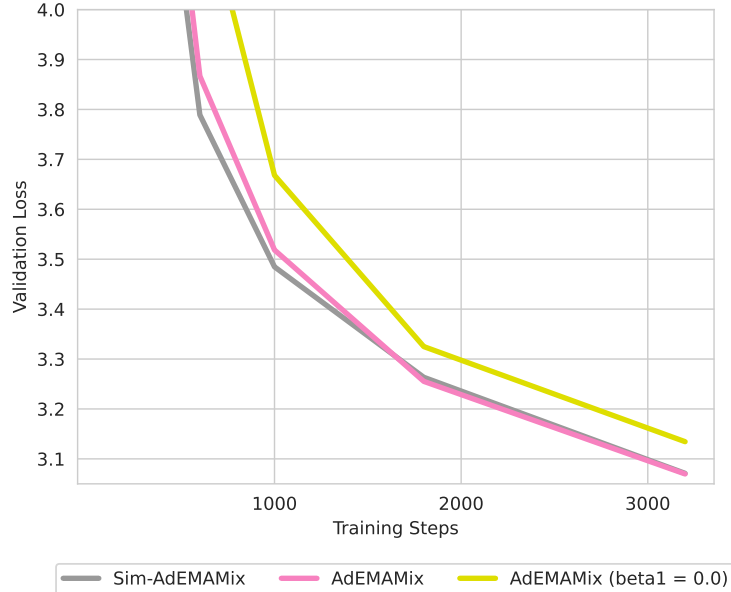


Figure 3: Best runs of AdEMAMix (with and without $\beta_1 = 0.0$) and our Simplified-AdEMAMix at the higher batch size. This supports our fifth contribution: Simplified-AdEMAMix matches full AdEMAMix with a single momentum term. Experimental details are in Section 5.1.

Adam with momentum scheduling—so, as claimed in the third contribution, plain Adam with a scheduled momentum already recovers AdEMAMix’s large-batch performance.

6 CONCLUSION

This paper delivered a theory, its validation, and a resulting algorithm. On the theory side, we established explicit connections between accelerated SGD variants and several recently proposed optimizers—Schedule-Free methods, AdEMAMix, MARS, and Lion—showing they form a single, well-understood family (contribution 1). Empirically, we confirmed the framework’s predictions on a 150m language model: AdEMAMix, the closest match to accelerated SGD, performs best at small batch sizes (contribution 2); the benefit vanishes at large batch sizes, where scheduled-momentum Adam already matches AdEMAMix (contribution 3); and Schedule-Free AdamW’s large-batch shortfall traces to its coupled coefficients rather than its preconditioning order (contribution 4).

Turning the theory into practice, we introduced Simplified-AdEMAMix, which drops one of AdEMAMix’s two momentum buffers yet matches its performance across both small and large batch sizes—removing AdEMAMix’s extra memory overhead at no measured cost in quality (contribution 5). Taken together, these results show that reading modern optimizers through the lens of accelerated SGD is not only explanatory but also constructive: the same lens that unifies them yields a simpler method.

REFERENCES

- Chen, X., Liang, C., Huang, D., Real, E., Wang, K., Pham, H., Dong, X., Luong, T., Hsieh, C., Lu, Y., and Le, Q. V. Symbolic discovery of optimization algorithms. In Oh, A., Naumann, T., Globerson, A., Saenko, K., Hardt, M., and Levine, S. (eds.), *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023*, 2023. URL http://papers.nips.cc/paper_files/paper/2023/hash/9a39b4925e35cf447ccba8757137d84f-Abstract-Conference.html.

- Defazio, A. Momentum via primal averaging: Theoretical insights and learning rate schedules for non-convex optimization, 2021. URL <https://arxiv.org/abs/2010.00406>.
- Defazio, A., Yang, X., Mehta, H., Mishchenko, K., Khaled, A., and Cutkosky, A. The road less scheduled. *CoRR*, abs/2405.15682, 2024. doi: 10.48550/ARXIV.2405.15682. URL <https://doi.org/10.48550/arXiv.2405.15682>.
- Gupta, K., Siegel, J., and Wojtowysch, S. Achieving acceleration despite very noisy gradients, 2023. URL <https://arxiv.org/abs/2302.05515>.
- Jain, P., Kakade, S. M., Kidambi, R., Netrapalli, P., and Sidford, A. Accelerating stochastic gradient descent for least squares regression. In Bubeck, S., Perchet, V., and Rigollet, P. (eds.), *Proceedings of the 31st Conference On Learning Theory*, volume 75 of *Proceedings of Machine Learning Research*, pp. 545–604. PMLR, 06–09 Jul 2018. URL <https://proceedings.mlr.press/v75/jain18a.html>.
- Kidambi, R., Netrapalli, P., Jain, P., and Kakade, S. M. On the insufficiency of existing momentum schemes for stochastic optimization. In *International Conference on Learning Representations*, 2018. URL <https://openreview.net/forum?id=rJTutzbA->.
- Liu, C. and Belkin, M. Accelerating sgd with momentum for over-parameterized learning. In *International Conference on Learning Representations*, 2020. URL <https://openreview.net/forum?id=rlgixp4FPH>.
- Lucas, J., Sun, S., Zemel, R., and Grosse, R. Aggregated momentum: Stability through passive damping. In *International Conference on Learning Representations*, 2019. URL <https://openreview.net/forum?id=Syxt5oC5YQ>.
- Ma, J. and Yarats, D. Quasi-hyperbolic momentum and adam for deep learning. In *International Conference on Learning Representations*, 2019. URL <https://openreview.net/forum?id=SlfUpoR5FQ>.
- Pagliardini, M., Ablin, P., and Grangier, D. The ademamix optimizer: Better, faster, older. 2024. URL <https://arxiv.org/abs/2409.03137>.
- Porian, T., Wortsman, M., Jitsev, J., Schmidt, L., and Carmon, Y. Resolving discrepancies in compute-optimal scaling of language models. *CoRR*, abs/2406.19146, 2024. doi: 10.48550/ARXIV.2406.19146. URL <https://doi.org/10.48550/arXiv.2406.19146>.
- Vaswani, S., Bach, F., and Schmidt, M. Fast and faster convergence of sgd for over-parameterized models and an accelerated perceptron. In Chaudhuri, K. and Sugiyama, M. (eds.), *Proceedings of the Twenty-Second International Conference on Artificial Intelligence and Statistics*, volume 89 of *Proceedings of Machine Learning Research*, pp. 1195–1204. PMLR, 16–18 Apr 2019. URL <https://proceedings.mlr.press/v89/vaswani19a.html>.
- Yuan, H., Liu, Y., Wu, S., Zhou, X., and Gu, Q. Mars: Unleashing the power of variance reduction for training large models, 2024. URL <https://arxiv.org/abs/2411.10438>.
- Ziyin, L., Wang, Z. T., and Ueda, M. Laprop: Separating momentum and adaptivity in adam, 2021. URL <https://arxiv.org/abs/2002.04839>.

A HYPERPARAMETERS

For reproducibility of the empirical validation behind our second, third, fourth, and fifth contributions, we list the full sweeps below, beginning with the small-batch experiments.

1. AdamW with cosine decay - 51.2k warmup - learning rate in $[3.16e-4, 1e-3, 3.16e-3]$, β_1 in $[0.9, 0.95]$, β_2 in $[0.99, 0.999, 0.99968, 0.9999]$. The optimal values of β_1 and β_2 were .9 and .999 respectively matching the default values. We note that for larger batch sizes it is common to use $\beta_2 = .95$, the benefit of higher β_2 at smaller batch sizes has also been observed by Porian et al. (2024).
2. AdamW with cosine decay - 10k warmup - learning rate in $[3.16e-4, 1e-3, 3.16e-3]$, $\beta_1 = 0.9$, $\beta_2 = 0.999$ i.e. we fix β_1, β_2 to be the optimal values from the previous sweep. This performed worse than warmup of 51.2k steps.
3. AdamW constant fraction weight averaging: - learning rate in $[3.16e-4, 1e-3, 3.16e-3]$, $\beta_1 = 0.9$, β_2 in $[0.99, 0.997, 0.999, 0.9997]$, δ in $[0.05, 0.1, 0.2]$.
4. AdamW with cosine decay and weight averaging - learning rate in $[3.16e-4, 1e-3, 3.16e-3]$, $\beta_1 = 0.9$, $\beta_2 = 0.999$, δ in $[0.025, 0.05, 0.1]$.
5. Accelerated SGD based AdamW with cosine decay - learning rate in $[3.16e-4, 1e-3, 3.16e-3]$, β_1 in $[0.999, 0.99968, 0.9999]$, β_2 in $[0.99, 0.9968, 0.999]$, $\beta_3 = 0.9$
6. Accelerated SGD based AdamW with constant learning rate and weight averaging - learning rate in $[3.16e-4, 1e-3, 3.16e-3]$, β_1 in $[0.99684, 0.999]$, β_2 in $[0.999]$, $\beta_3 = 0.9$, δ in $[0.05, 0.1]$
7. Accelerated SGD based AdamW with cosine decay and weight average - learning rate in $[3.16e-4, 1e-3, 3.16e-3]$, β_1 in $[0.99684, 0.999]$, $\beta_2 = 0.999$, δ in $[0.05, 0.1]$, $\beta_3 = 0.9$
8. Schedulefree AdamW with constant learning rate - learning rate in $[3.16e-4, 1e-3, 3.16e-3, 1e-2]$, β_1 in $[0.8, 0.9, 0.95]$, $\beta_2 = 0.999$
9. Schedulefree AdamW with cosine decay - $[3.16e-4, 1e-3, 3.16e-3, 1e-2]$, β_1 in $[0.8, 0.9, 0.95]$, $\beta_2 = 0.999$
10. MARS - $[3.16e-4, 1e-3, 3.16e-3, 1e-2]$, β_1 in $[0.9, 0.95, 0.99]$, β_2 in $[0.99, 0.999]$, γ in $[0.0, 0.01, 0.02, 0.03, 0.04, 0.05]$, precondition 1d was set to True.
11. AdEMAMix - $[3.16e-4, 1e-3, 3.16e-3]$, β_1 in $[0.0, 0.9]$, $\beta_2 = 0.999$, β_3 in $[0.99, 0.999, 0.9999]$, α in $[2, 4, 8, 16]$.
12. Sim-AdEMAMix - $[1e-6, 3.16e-6, 1e-5, 3.16e-5]$, β_1 in $[0.99, 0.999, 0.9999]$, $\beta_2 = 0.999$, α in $[10, 20, 50, 100]$

The hyperparameter sweeps for the large batch experiments are provided below:

1. Schedule-Free AdamW: $[1e-3, 3.16e-3, 1e-2]$, β in $[0.8, 0.9, 0.95]$, β_2 in $[0.9, 0.95]$, r in $[0.0, 5.0, 9.0, 50.0]$
2. AdamW: $[1e-3, 3.16e-3, 1e-2]$, β_1 in $[0.9, 0.95]$, β_2 in $[0.9, 0.95]$
3. LProp: $[1e-3, 3.16e-3, 1e-2]$, β_1 in $[0.9, 0.95]$, β_2 in $[0.9, 0.95]$
4. AdEMAMix: $[1e-3, 3.16e-3, 1e-2]$, β_1 in $[0.0, 0.9]$, $\beta_2 = 0.95$, β_3 in $[0.9, 0.95, 0.99]$, α in $[2, 4, 8, 16]$
5. Sim-AdEMAMix: $[1e-4, 3.16e-4, 1e-3]$, β_1 in $[0.9, 0.95, 0.99]$, $\beta_2 = 0.95$, α in $[0.0, 0.5, 1.0]$

B EQUIVALENCE OF PREVIOUS ACCELERATION METHODS

This appendix backs the claim, made in the Background and used throughout, that a single form captures the accelerated SGD literature. Starting from the general accelerated SGD form of Equation (1), we show that every method of Jain et al. (2018); Vaswani et al. (2019); Liu & Belkin (2020); Gupta et al. (2023) can be rewritten to fit it. These reductions are what license us to treat Equation (1) as the common target for all the equivalences in the main text.

B.1 AGNES

The update for Gupta et al. (2023) is given below:

$$x'_n = x_n + \alpha v_n \quad x_{n+1} = x'_n - \eta g'_n \quad v_{n+1} = \rho_n(v_n - g'_n)$$

where g'_n is stochastic gradient evaluated on x'_n and the final function is evaluated on x_n . The above equations can be rewritten as

$$x'_{n+1} = x'_n - \eta g'_n + \alpha v_{n+1} \quad -\frac{v_{n+1}}{\rho_n} = \rho_{n-1} \left(-\frac{v_n}{\rho_{n-1}} \right) + g'_n$$

Thus x'_{n+1} follows update equation of the form of Equation (1).

B.2 ASGD

The update for Jain et al. (2018) is given by:

$$y_{j-1} = \alpha x_{j-1} + (1 - \alpha)v_{j-1} \quad x_j = y_{j-1} - \delta g_{j-1} \quad z_{j-1} = \beta y_{j-1} + (1 - \beta)v_{j-1} \quad v_j = z_{j-1} - \gamma g_{j-1}$$

where g_{j-1} represents the stochastic gradient evaluated on y_{j-1} and the function is evaluated on the tail averaged x .

The update equations above can be rewritten as:

$$y_j = y_{j-1} - \alpha \delta g_{j-1} - (1 - \alpha)[y_{j-1} - v_j] \quad \frac{y_{j-1} - v_j}{\gamma - (1 - \beta)\alpha\delta} = (1 - \beta)\alpha \frac{y_{j-2} - v_{j-1}}{\gamma - (1 - \beta)\alpha\delta} + g_{j-1}$$

The update equations above follow the form of Equation (1).

B.3 MASS

The update for Liu & Belkin (2020) is given by:

$$w_{t+1} = u_t - \eta_1 g_t \quad u_{t+1} = (1 + \gamma)w_{t+1} - \gamma w_t + \eta_2 g_t$$

where g_t is the stochastic gradient evaluated on u_t and the function is evaluated on w_t . These equations can be rewritten as:

$$u_{t+1} = u_t - \gamma(w_t - u_t) - [\eta_1(1 + \gamma) - \eta_2]g_t \quad \frac{w_t - u_t}{\eta_1\gamma - \eta_2} = \gamma \frac{w_{t-1} - u_{t-1}}{\eta_1\gamma - \eta_2} + g_t$$

The update equations above follow the form of Equation (1).

B.4 SGD WITH NESTEROV ACCELERATION

The update for Vaswani et al. (2019) is given by:

$$w_{k+1} = \zeta_k - \eta g_k \quad \zeta_k = \alpha_k v_k + (1 - \alpha_k)w_k \quad v_{k+1} = \beta_k v_k + (1 - \beta_k)\zeta_k - \gamma_k \eta g_k$$

These equations can be rewritten as:

$$\zeta_{k+1} = \zeta_k - \eta g_k + \alpha_{k+1}[v_{k+1} - w_{k+1}]; \quad v_{k+1} - w_{k+1} = \beta_k(1 - \alpha_k)[v_k - w_k] - \eta(\gamma_k - 1)g_k$$